CS 112

Introduction to Data Structures

WEEK 08

Algorithm Analysis and Big-Oh

Eric Araújo

Calvin University · Fall 2026

This Week

1. How long does each operation take?
2. Big-Oh notation
3. The operations, in Big-Oh **KEY**
4. $a \cdot f(n) + b$
5. Reading it off the code
6. Why it matters: electricity
7. `operator<<` for a container

Vector Subscript

```
v[i]    →    *(v.myArray + i * sizeof(Item))
```

3 operations, whatever *i* is and however big the vector is: **constant time**.

Vector Append

Room left

```
myArray[mySize] = it;
++mySize;
```

a few operations: **constant**

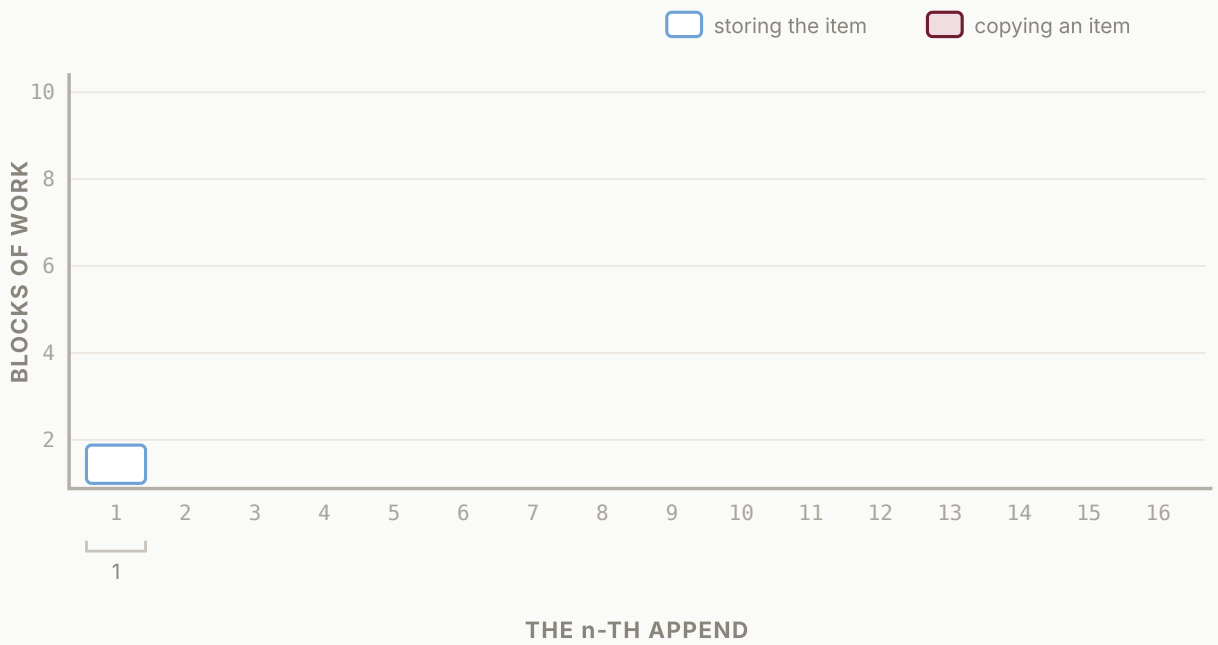
Full

```
new Item[2 * mySize]
copy n items
delete [] old
```

about 3*n* + 5: **linear**

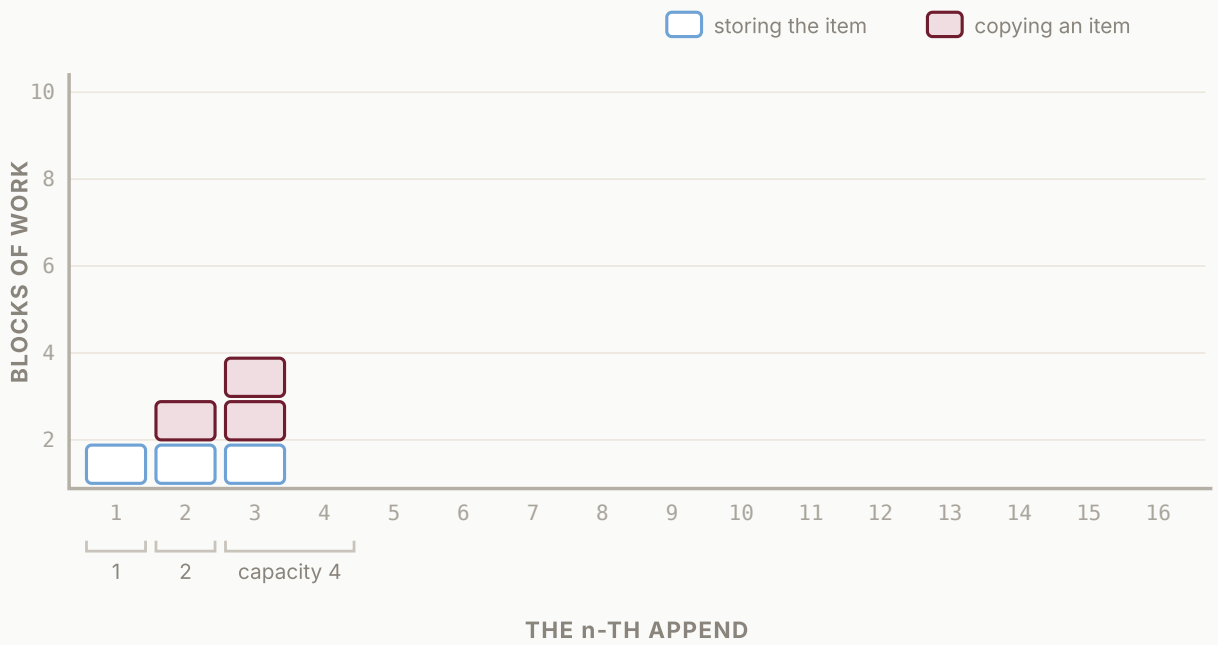
Vector Append, on Average

1. Append 1: there is room, so it just stores the item.



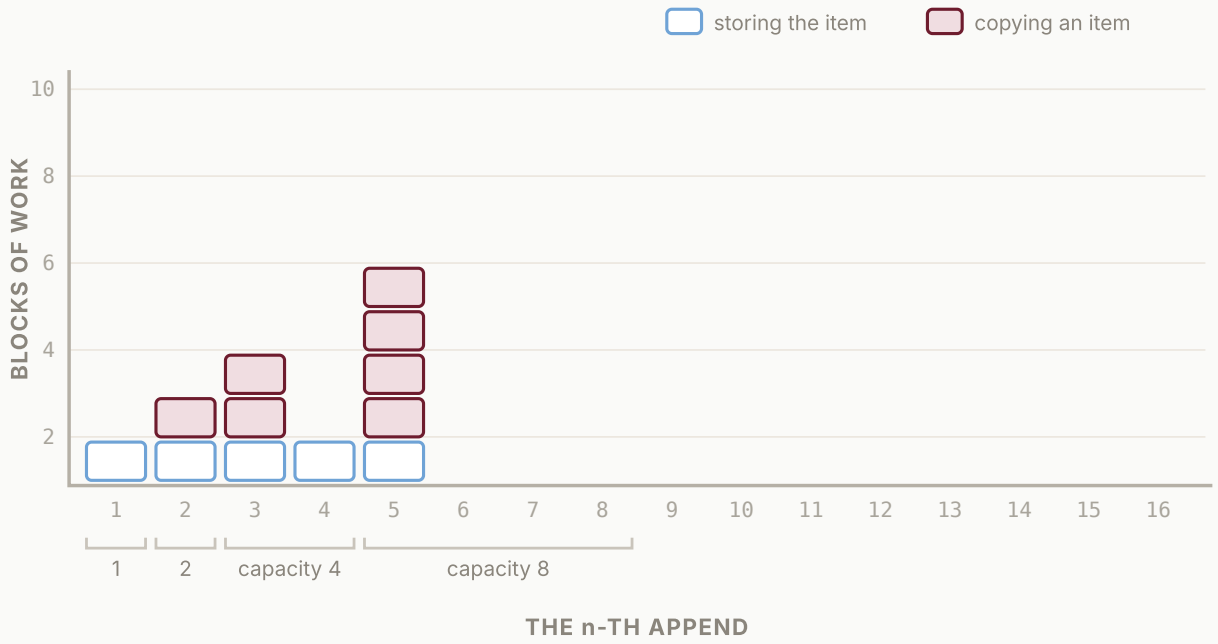
Vector Append, on Average

2. Append 3: the array is full, so it copies 2 items first.



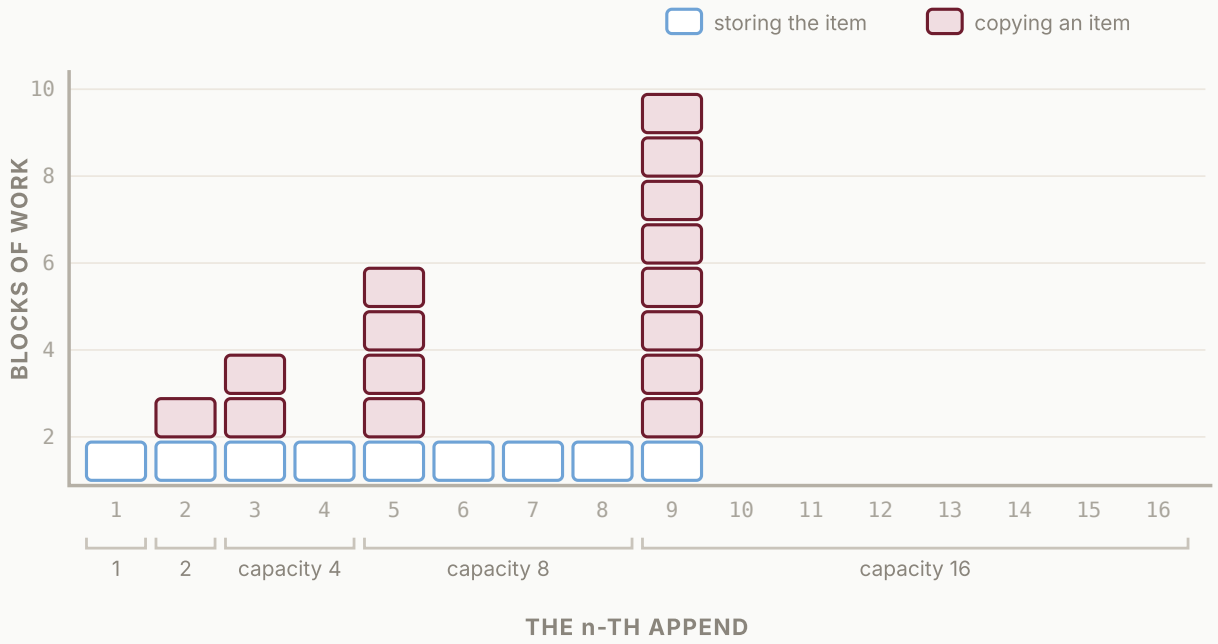
Vector Append, on Average

3 Append 5: the array is full, so it copies 4 items first.



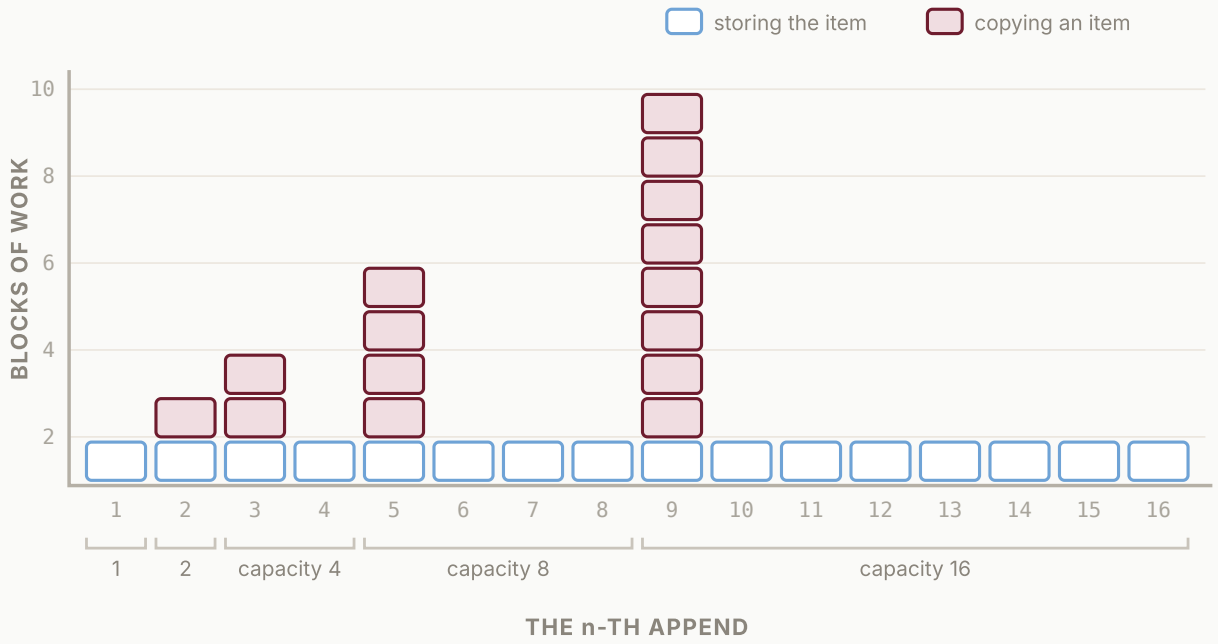
Vector Append, on Average

4 Append 9: the array is full, so it copies 8 items first.



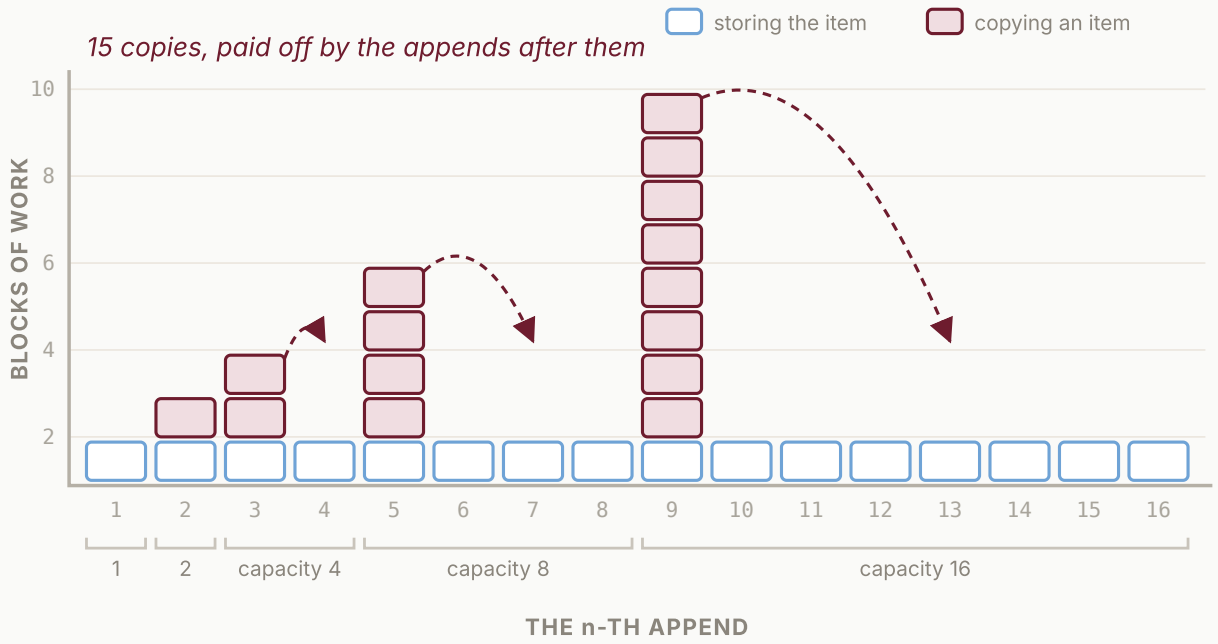
Vector Append, on Average

5 Append 16: there is room, so it just stores the item.



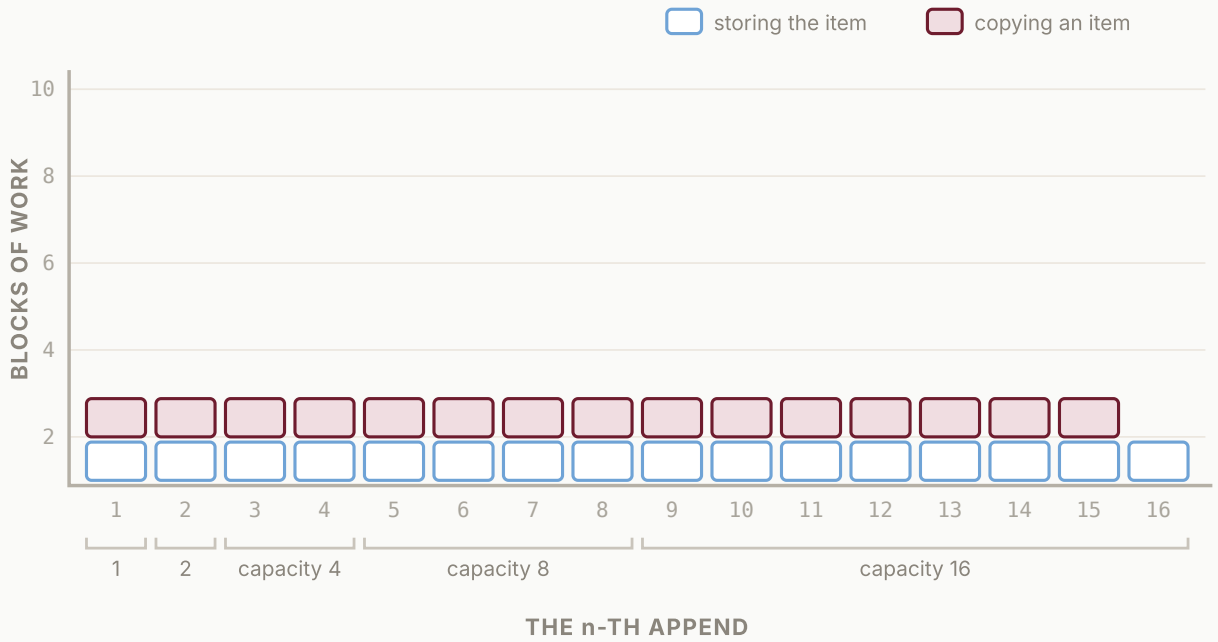
Vector Append, on Average

6 Each spike is paid for by the appends that follow it.



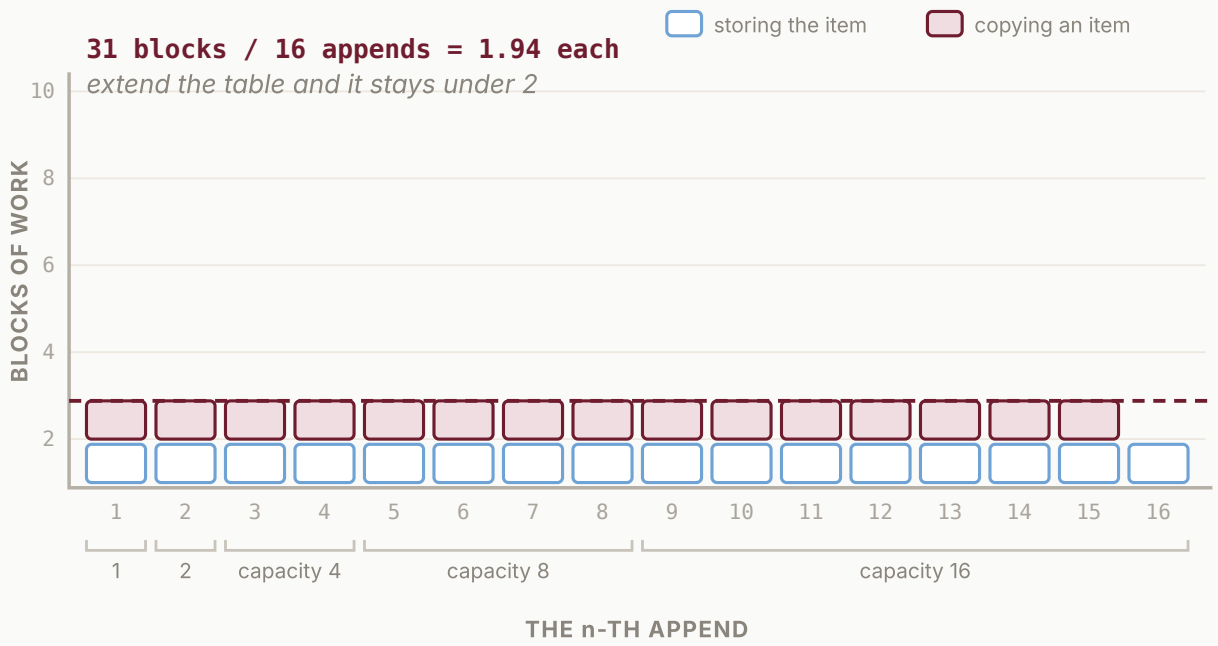
Vector Append, on Average

7 The same blocks, laid flat. Nothing was added or removed.



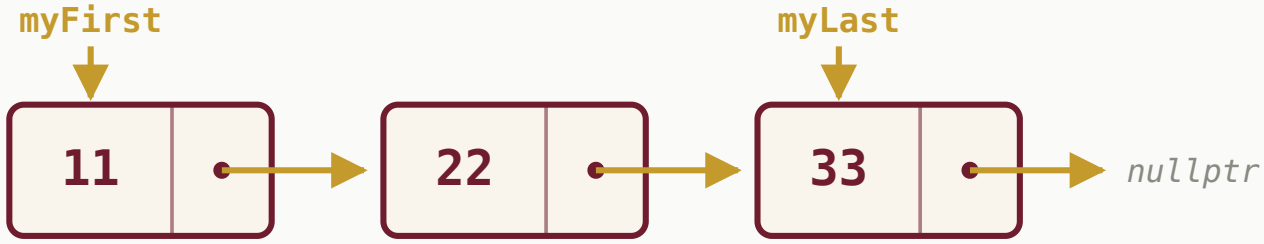
Vector Append, on Average

8 31 blocks over 16 appends, and the average does not grow with n.



Amortized constant time.

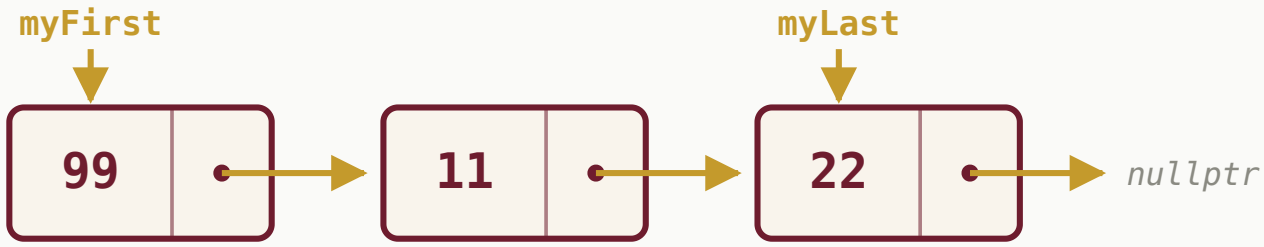
List Append




```
Node* n = new Node(it, nullptr);
if (mySize == 0) myFirst = n;
else             myLast->myNext = n;
myLast = n;
++mySize;
```

About 6 operations, empty list or not: **constant**.

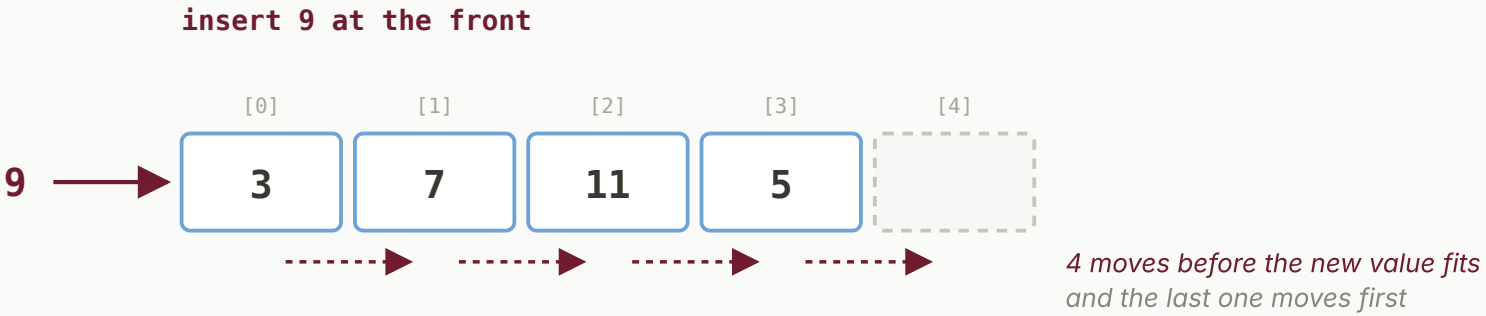
List Prepend




```
myFirst = new Node(it, myFirst);
if (mySize == 0) myLast = myFirst;
++mySize;
```

About 5 operations, empty list or not: **constant**.

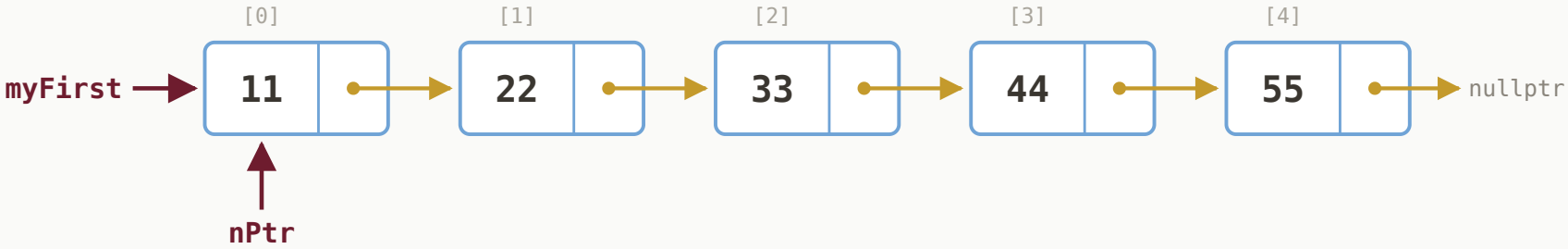
Vector Prepend



Every item shifts first: **linear**. So `vector` has no `push_front`.

List Subscript

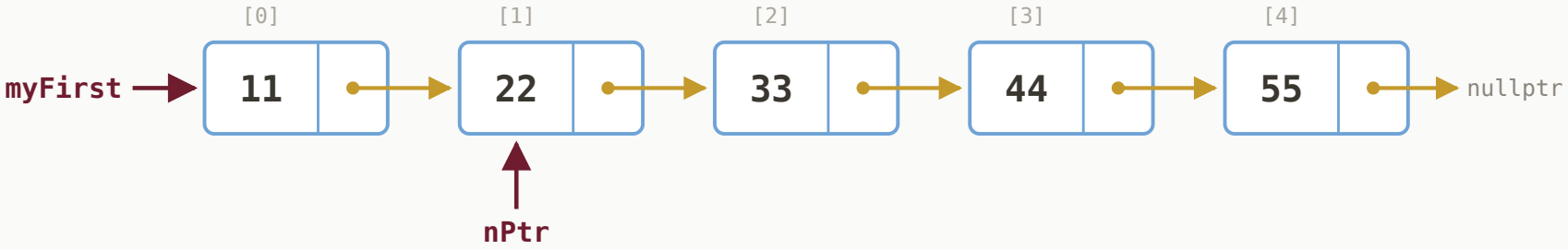
1. l[3] starts where every walk starts: nPtr = myFirst.




```
Item& List::getIndex(unsigned ix) {
  Node* nPtr = myFirst;
  for (unsigned i = 0; i < ix; i++)
    nPtr = nPtr->myNext;
  return nPtr->myItem;
}
```

List Subscript

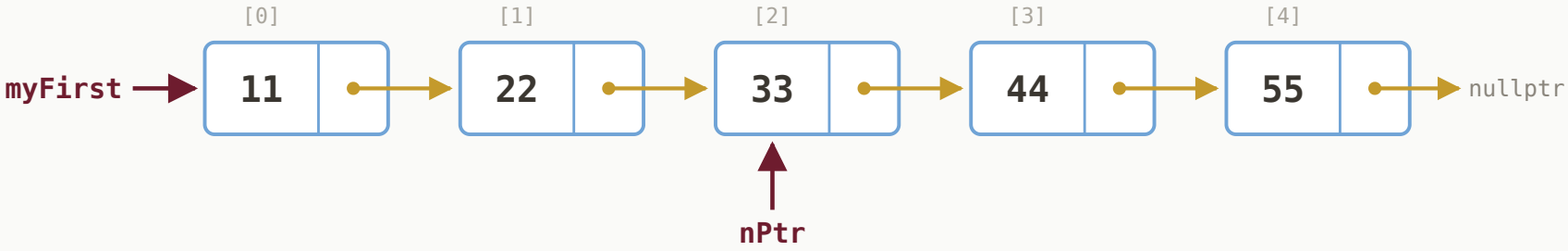
2 One pass of the loop, one hop: nPtr = nPtr→myNext.




```
Item& List::getIndex(unsigned ix) {
  Node* nPtr = myFirst;
  for (unsigned i = 0; i < ix; i++)
    nPtr = nPtr->myNext;
  return nPtr->myItem;
}
```

List Subscript

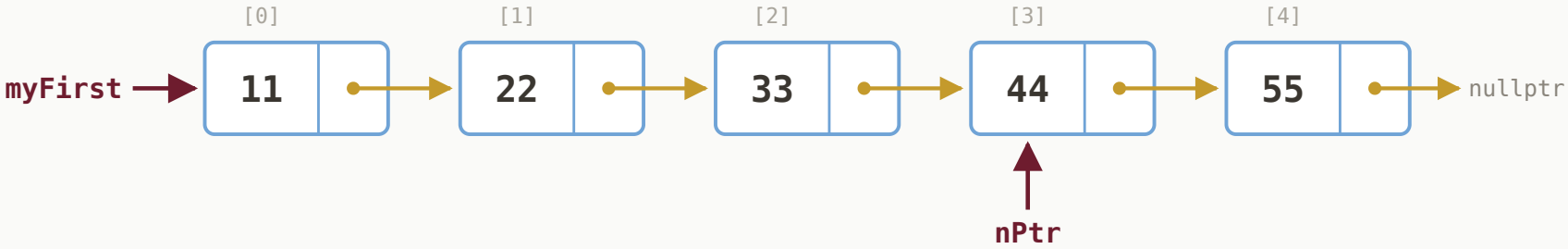
3. Another hop. There is no way to jump ahead.




```
Item& List::getIndex(unsigned ix) {
  Node* nPtr = myFirst;
  for (unsigned i = 0; i < ix; i++)
    nPtr = nPtr->myNext;
  return nPtr->myItem;
}
```

List Subscript

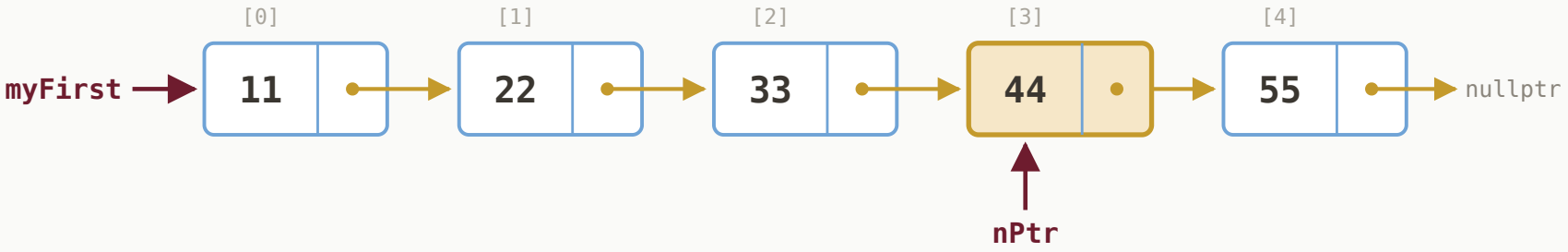
4 The third hop reaches item 3, and i == 3 ends the loop.




```
Item& List::getIndex(unsigned ix) {
  Node* nPtr = myFirst;
  for (unsigned i = 0; i < ix; i++)
    nPtr = nPtr->myNext;
  return nPtr->myItem;
}
```

List Subscript

5 Three hops for index 3. The last item takes n - 1, so on average the loop runs about n/2 times: linear.




```
Item& List::getIndex(unsigned ix) {
  Node* nPtr = myFirst;
  for (unsigned i = 0; i < ix; i++)
    nPtr = nPtr->myNext;
  return nPtr->myItem;
}
```

About *n*/2 hops on average: **linear**. So STL's `list` has no `[]`.

TALK TO YOUR NEIGHBOR · TTYN

Which of these is linear?

A. `v[500]` on a vector of 1000

B. `push_front` on a list of 1000

C. inserting at the front of a vector of 1000

D. `push_back` on a list of 1000

Big-Oh Notation

Complexity	Big-Oh	
constant	O(1)	polynomial
logarithmic	O(lg n)	
linear	O(n)	
quadratic	$O(n^2)$	
cubic	$O(n^3)$	
exponential	$O(2^n)$	**non-polynomial**
factorial	O(n!)	

Better at the top, worse at the bottom.

The Operations, in Big-Oh

Operation	Complexity	Big-Oh
vector subscript	constant	O(1)
vector append	amortized constant	**O(1*)**
vector prepend	linear	O(n)
list append	constant	O(1)
list prepend	constant	O(1)
list subscript	linear	O(n)
shallow copy	constant	O(1)
deep copy, assignment	linear	O(n)
input or output	linear	O(n)

Need an operation the container doesn't have? Choose a different container.

What O(f(n)) Means

$$\text{exact cost} = a \cdot f(n) + b$$

> As *n* grows, f(n) dominates. Nobody cares about *a* and *b*.

At a Million Items

Operation	Exact cost	At n = 1,000,000	Big-Oh
vector subscript	3	3	O(1)
list append	5	5	O(1)
vector prepend	$3n + 2$	3,000,002	O(n)
list subscript	$0.5n + 2$	500,002	O(n)

> 3 and 5 are both "about 1" next to a million. 3,000,002 and 500,002 are both "about a million".

TALK TO YOUR NEIGHBOR · TTYN

An algorithm takes `4n + 200` steps. Its complexity is...

A. O(4n)

B. O(n)

C. O(n + 200)

D. O(200)

Reading It Off the Code

```
for (int i = 0; i < n; ++i) {
  total += a[i];
}
```

one loop over n: **O(n)**

```
for (int i = 0; i < n; ++i) {
  for (int j = 0; j < n; ++j) {
    if (a[i] == a[j]) ...
  }
}
```

a loop inside a loop: **O(n^2)**

> Loops one after another add, keep the biggest. Nested loops multiply.

TALK TO YOUR NEIGHBOR · TTYN

What is the complexity of this loop?

```
for (int i = 1; i < n; i = i * 2) {
    cout << i << endl;
}
```

A. O(n)

B. $O(n^2)$

C. O(lg n)

D. O(n lg n)

Searching a Million Items

Search	Big-Oh	Comparisons
linear search, any vector	O(n)	up to 1,000,000
binary search, sorted vector	O(lg n)	about 20

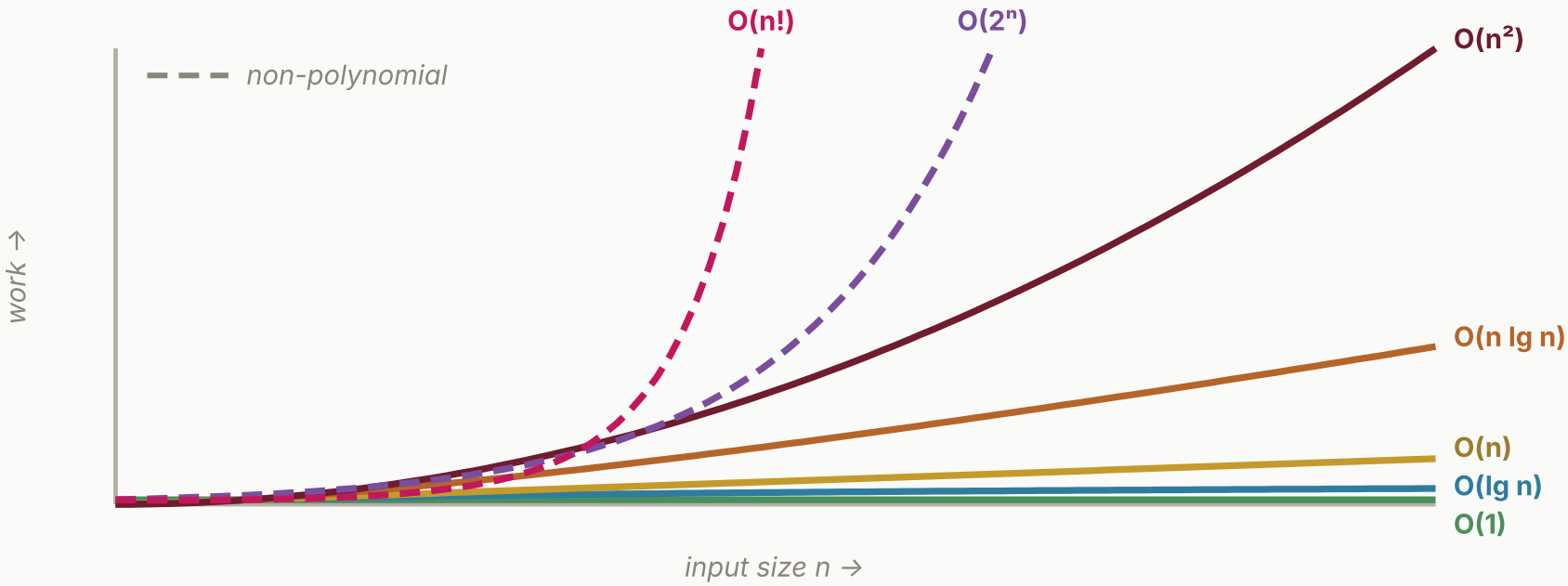
> Computers run on electricity. Fewer comparisons, less electricity, less carbon.

Sorting a Million Items

Sort	Big-Oh	Steps, roughly
bubble sort	$O(n^2)$	1,000,000,000,000
quicksort	O(n lg n)	20,000,000

> Big-Oh lets us estimate an algorithm's **carbon footprint**.

Visualizing Big-Oh



TALK TO YOUR NEIGHBOR · TTYN

A is $O(n^2)$, B is $O(n \lg n)$. Which should you use?

A. Always B

B. Always A

C. B for large n; for small n, A may be faster

D. Whichever is easier to write

operator<<

```
cout << aList << endl;

// is really
operator<<( operator<<(cout, aList), endl );
```

TALK TO YOUR NEIGHBOR · TTYN

What should `operator<<(ostream& out, const List& list)` return?

A. nothing, it is `void`

B. `list`

C. `out`

D. `true`

operator<<

```
ostream& operator<<(ostream &out, const List &list) {
    list.writeTo(out);
    return out;
}
```

> `out` is not `const`: writing to it changes it.

Week 08 Recap

- Big-Oh describes **how work grows with n**, not seconds on a clock
- exact cost = $a \cdot f(n) + b$; keep f(n)
- Vector append is `O(1*)`, amortized constant
- Nested loops multiply; halving the problem gives `lg n`
- Better algorithms use less electricity

Next: a09 →

This deck stands on earlier CS112 materials by
Joel Adams and **Victor Norman** · adapted and extended by **Eric Araújo**